

Dockerfile Avanzado

Ejercicios Prácticos

Arturo Silvelo

Try New Roads

Ejercicios

Estos ejercicios prácticos te permitirán aplicar las técnicas avanzadas de Dockerfile aprendidas en los ejemplos. Trabajarás con aplicaciones reales:

- **Backend:** API REST desarrollada con **NestJS** (TypeScript)
- **Frontend:** Aplicación web desarrollada con **Angular** (TypeScript)

Cada ejercicio incluye un **Dockerfile base sin optimizar** que deberás mejorar aplicando las técnicas aprendidas.

Ejercicio 1: Backend NestJS

Objetivo

Optimizar progresivamente el Dockerfile del backend NestJS aplicando cada una de las técnicas vistas en el temario.

1.1 Dockerfile Base

Analiza el `Dockerfile` sin optimizar ubicado en `backend/Dockerfile`.

- **Comando para construir:**

```
docker build -f backend/Dockerfile -t backend-base backend
```

- Servicio funciona

```
curl http://localhost:3000/api/healthcheck
```

1.2 Multi-stage Build

Crea `Dockerfile.multistage` que implemente multi-stage builds:

- **Etapas 1 (build):** Compila la aplicación TypeScript
 - `npm run build`
- **Etapas 2 (test):** Ejecuta los tests sobre el código compilado
 - `npm test`
- **Etapas 3 (production):** Solo archivos necesarios para ejecutar

- Construcción

```
docker build -f Dockerfile.multistage -t backend-multistage backend
```

- Verificación

```
docker images --filter reference="backend*"
```

1.3 Optimización de Capas

Crea `Dockerfile.optimized` mejorando el anterior:

- Agrupa comandos RUN relacionados
- Limpia archivos temporales en la misma capa
- Reordena instrucciones para mejor cache

- Construcción

```
docker build -f Dockerfile.optimized -t backend-optimized backend
```

- Verificación

```
docker image ls backend*
```

1.4 Variables ARG y ENV

Crea `Dockerfile.variables` que gestione correctamente las variables:

- Usa **ARG** para las variables de entorno.

- Construcción

```
docker build -f Dockerfile.variables -t backend-variables backend
```

- Verificación

```
docker run --rm --init backend-variables env  
docker run --rm --init -p 3000:8000 -e PORT=8000 backend-variables
```

1.5 Gestión Segura de Secretos

Crea `Dockerfile.secure` que elimine todos los secretos hardcodeados:

- Variables sensibles se pasan solo en runtime (`-e`)
- Opción para leer secretos desde archivos montados

- Construcción

```
docker build -f Dockerfile.secure -t backend-secure backend
```

- Verificación

```
docker run --rm --init backend-secure env
```

1.6 Gestión de usuario

Crea `Dockerfile.no-root` que utilice un usuario no root.

- Construcción

```
docker build -f Dockerfile.no-root -t backend-no-root backend
```

- Verificación

```
docker run --rm --init backend-no-root whoami
```